

1. **What is ReactJS?**

ReactJS is a JavaScript library used to build user interfaces.

2. **Does React use HTML?**

No, React uses **JSX**, which looks similar to HTML.

3. **When was React first released?**

React was first released in **March 2013**.

4. **Give two drawbacks of React.**

- React is only a UI library, not a complete framework.
- Learning JSX and state management can be challenging.

5. **Difference between Real DOM and Virtual DOM?**

- **Real DOM:** Updates the entire DOM, slower.
- **Virtual DOM:** Updates only changed parts, faster.

6. **What is Flux Concept in React?**

Flux is an architecture that follows **one-way data flow**.

7. **Define Redux in React.**

Redux is a state management library used to manage application state.

8. **What is the Store feature in Redux?**

Store is a central place where the entire application state is stored.

9. **What is an Action in Redux?**

An Action is an object that describes what happened in the application.

10. **Name important features of React.**

- Component-based architecture
- Virtual DOM
- JSX
- One-way data binding
- Reusable components

11. **Explain Stateless Components.**

Stateless components are components that do not manage their own state.

12. **Explain React Router.**

React Router is a library used for navigation between pages in React applications.

13. Explain JSX.

JSX is a syntax extension that allows writing HTML-like code inside JavaScript.

14. What is Dispatcher?

Dispatcher is a central hub in Flux that sends actions to stores.

15. What is a Callback Function? What is its purpose?

A callback function is executed after another function completes, often used after `setState()`.

16. Explain High Order Component (HOC).

HOC is a function that takes a component and returns an enhanced component.

17. Explain the Presentational Component.

Presentational components focus only on UI and receive data through props.

18. What are Props in ReactJS?

Props are read-only data passed from a parent component to a child component.

18) Explain yield catchphrase in JavaScript

yield is a keyword used inside **generator functions** to pause and resume execution.

19) Name two types of React components

- **Functional Components**
- **Class Components**

20) Explain Synthetic Event in ReactJS

A Synthetic Event is a cross-browser wrapper around the browser's native event, providing a consistent event handling API.

21) What is React State?

State is an object that stores data or information that can change over time and affects how a component renders.

22) How can you update state in ReactJS?

State can be updated using:

- `setState()` in Class Components
- `useState()` setter function in Functional Components

Example:

```
setCount(count + 1);
```

23) Explain the use of the Arrow Function in React

Arrow functions are used to write shorter code and automatically bind this to the current context.

Example:

```
const handleClick = () => {  
  console.log("Clicked");  
};
```

24) State the main difference between Props and State

Props	State
Read-only (Immutable)	Can be changed (Mutable)
Passed from parent component	Managed within the component

25) Explain Pure Components in ReactJS

Pure Components are components that re-render only when their props or state change, improving performance.

26) What kind of information controls a component in React?

A component is controlled by:

- **Props** (data from parent)
- **State** (internal data)

27) What is create-react-app?

create-react-app is a command-line tool used to quickly create and set up a new React application without manual configuration.

Example:

```
npx create-react-app my-app
```

28) Explain the use of 'key' in React list

The key prop uniquely identifies list items, helping React efficiently update, add, or remove items.

Example:

```
{users.map(user => (  
  <li key={user.id}>{user.name}</li>  
))}
```

29) What are Children Props?

children is a special prop that allows components to pass and render other components or elements inside them.

Example:

```
<Card>  
  <h1>Hello</h1>  
</Card>
```

30) Explain Error Boundaries

Error Boundaries are React components that catch JavaScript errors in child components and display a fallback UI instead of crashing the app.

31) What is the use of Empty Tags (<> </>) ?

Empty tags are used to create **Fragments**, allowing multiple elements to be returned without adding extra DOM nodes.

Example:

```
<>  
  <Header />  
  <Footer />  
</>
```

32) Explain Strict Mode

<React.StrictMode> is a tool that helps identify potential problems in an application during development.

Example:

```
<React.StrictMode>  
  <App />  
</React.StrictMode>
```

33) What are React Portals?

React Portals allow rendering a component outside its parent DOM hierarchy.

Commonly used for **modals, popups, and tooltips**.

34) What is Context?

Context provides a way to share data across components without passing props manually at every level.

Example: Theme, User Authentication, Language.

35) What is the use of Webpack?

Webpack is a module bundler that bundles JavaScript, CSS, images, and other assets into optimized files.

36) What is Babel in ReactJS?

Babel is a JavaScript compiler that converts modern JavaScript and JSX into browser-compatible JavaScript.

37) How can a browser read JSX files?

Browsers cannot understand JSX directly. Babel transpiles JSX into regular JavaScript that browsers can execute.

38) What are the major issues of using MVC architecture in React?

- Complex application structure
 - Expensive DOM manipulation
 - Reduced performance in large applications
-

39) What can be done when there is more than one line of expression?

Use **multi-line JSX expressions** by wrapping them inside parentheses ().

Example:

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </div>  
);
```

40) What is Reduction?

Reduction refers to the process of managing and updating state using **Reducers** in React or Redux.

Example:

```
const newState = reducer(state, action);
```

41) Explain Synthetic Events

Synthetic Events are React's cross-browser wrappers around native browser events, providing a consistent API.

Examples: onClick, onChange, onSubmit

42) When should you use Class Components instead of Function Components?

Use Class Components when you need lifecycle methods in older React versions. Nowadays, Hooks in Functional Components are preferred.

43) How can you share an element in parsing?

Data can be shared between components using:

- **Props**
 - **State**
 - **Context API**
-

44) Explain the term Reconciliation

Reconciliation is the process by which React compares the new Virtual DOM with the previous one and updates only the changed parts in the Real DOM.

45) How can you re-render a component without using `setState()`?

You can use:

- `forceUpdate()` in Class Components
- Updating parent props
- Changing Context values

Example:

```
this.forceUpdate();
```

46) Can you update Props in React?

No, props are **read-only (immutable)** and cannot be updated directly by a child component.

47) Explain the term 'Restructuring' (Destructuring).

Destructuring is a JavaScript feature used to extract values from arrays or objects into separate variables.

Example:

```
const { name, age } = user;
```

48) Can you update the values of Props?

No, props cannot be updated because they are immutable. Only the parent component can pass new props.

49) Explain the meaning of Mounting and Unmounting

- **Mounting:** The process of creating and inserting a component into the DOM.
- **Unmounting:** The process of removing a component from the DOM.

Example:

```
useEffect(() => {  
  console.log("Mounted");
```

```
  return () => {
```

```
    console.log("Unmounted");  
  };  
}, []);
```

50) What is the use of prop-types library?

prop-types is used for runtime type checking of props to ensure components receive the correct data type.

Example:

```
User.propTypes = {  
  name: PropTypes.string.isRequired  
};
```

51) What is React?

React is a JavaScript library for building fast, interactive, and reusable user interfaces.

52) What is useState() in React?

useState() is a React Hook used to manage state in functional components.

Example:

```
const [count, setCount] = useState(0);
```

53) What is JSX?

JSX (JavaScript XML) is a syntax extension that allows writing HTML-like code inside JavaScript.

Example:

```
const element = <h1>Hello React</h1>;
```

54) What are the differences between Controlled and Uncontrolled Components?

Controlled Component	Uncontrolled Component
----------------------	------------------------

Controlled by React state	Controlled by DOM
---------------------------	-------------------

Controlled Component	Uncontrolled Component
-----------------------------	-------------------------------

Uses useState	Uses useRef
Easier validation	Less code

55) When rendering a list, what is a key and what is its purpose?

A **key** is a unique identifier for list items. It helps React efficiently update, add, or remove items.

Example:

```
items.map(item => (  
  <li key={item.id}>{item.name}</li>  
))
```

56) Why do React Hooks make use of refs?

Refs (useRef) are used to:

- Access DOM elements directly.
- Store mutable values without causing re-renders.

Example:

```
const inputRef = useRef();  
inputRef.current.focus();
```

57) What are React Hooks?

Hooks are special functions that allow functional components to use React features like state and lifecycle methods.

Common Hooks:

- useState
- useEffect
- useContext
- useRef
- useMemo

- useCallback

58) Differentiate between State and Props

- | | |
|--------------------------------|--------------------------------|
| • State | • Props |
| • Managed within the component | • Passed from parent component |
| • Mutable (can change) | • Immutable (read-only) |
| • Used for dynamic data | • Used for passing data |
-

59) What is an Event in React?

- An event in React is an action triggered by the user or browser, such as a click, key press, or form submission.
- Example:
- `<button onClick={handleClick}>Click Me</button>`

- ---

60) What is Virtual DOM?

- Virtual DOM is a lightweight copy of the Real DOM that React uses to efficiently update the UI.

- ---

61) What is Context?

- Context is a feature that allows data to be shared across components without passing props manually at every level.

- ---

62) What is Children Prop?

- children is a special prop that allows components to receive and render nested elements.

- Example:

- ```
<Card>
 <h1>Hello</h1>
</Card>
```

- ---

### 63) What are Life Cycle Methods in React?

- Lifecycle methods are methods that execute at different stages of a component's life.
  - Main phases:
  - **Mounting** → componentDidMount()
  - **Updating** → componentDidUpdate()
  - **Unmounting** → componentWillUnmount()
- 

### 64) What are the limitations of React?

- React is only a UI library, not a complete framework.
  - JSX can be difficult for beginners.
  - Additional libraries are needed for routing and state management.
- 

### 65) What is the difference between Shadow DOM and Virtual DOM?

- |                             |                                  |
|-----------------------------|----------------------------------|
| • <b>Shadow DOM</b>         | • <b>Virtual DOM</b>             |
| • Browser technology        | • React feature                  |
| • Encapsulates HTML and CSS | • Improves rendering performance |
| • Used in Web Components    | • Used for efficient UI updates  |
- 

### 66) What is the difference between React and ReactDOM?

- |                             |                                       |
|-----------------------------|---------------------------------------|
| • <b>React</b>              | • <b>ReactDOM</b>                     |
| • Core React library        | • Renders React components to the DOM |
| • Used to create components | • Used to interact with browser DOM   |
- Example:
  - `import React from 'react';`  
`import ReactDOM from 'react-dom/client';`
- 

### 67) What is the purpose of the ReactTestUtils package?

- ReactTestUtils is used for testing React components and simulating user interactions.

- ---

#### 68) What are the core principles of Redux?

- **Single Source of Truth** – Entire state is stored in one store.
- **State is Read-Only** – State can only be changed using actions.
- **Changes are made with Pure Functions** – Reducers update the state.

- ---

#### 69) How to access Redux store outside a component?

- Export the Redux store and import it wherever needed.
- Example:
- import store from './store';

```
store.getState();
```

- ---

#### 70) What is the difference between React Context and React Redux?

- |                                   |                                   |
|-----------------------------------|-----------------------------------|
| • <b>Context API</b>              | • <b>Redux</b>                    |
| • Built into React                | • External library                |
| • Suitable for small/global state | • Suitable for large applications |
| • Simpler setup                   | • More powerful and scalable      |

- ---

#### 71) What is the difference between Component and Container in React Redux?

- **Component (Presentational Component):** Handles UI only.
- **Container Component:** Connects UI with Redux store and handles business logic.

- ---

#### 72) What is the difference between HTML and React Event Handling?

- **HTML**
- Uses lowercase events (onclick)
- Uses string values
- Example:
- **HTML**
- `<button onclick="myFunction()">Click</button>`
- **React**
- `<button onClick={myFunction}>Click</button>`
- **React**
- Uses camelCase (onClick)
- Uses functions as event handlers

### 73) What is Lifting State Up in React?

- Lifting State Up is the process of moving shared state to the closest common parent component so multiple child components can access it.

### 74) What are Fragments?

- Fragments allow you to group multiple elements without adding extra DOM nodes.
- Example:
- `<>`  
`<Header />`  
`<Footer />`  
`</>`

---

### 75) Why are Fragments better than container divs?

- No extra DOM elements are created.
- Improves performance.
- Keeps the DOM clean.

---

### 76) Why should we not update the state directly?

Directly updating state does not trigger a re-render. Always use `setState()` or state setter functions.

-  Wrong:
- `this.state.count = 1;`
-  Correct:
- `this.setState({ count: 1 });`
- ---

## 77) Differentiate React Hooks vs Classes

- |                                         |                            |
|-----------------------------------------|----------------------------|
| • <b>Hooks</b>                          | • <b>Classes</b>           |
| • Used in functional components         | • Used in class components |
| • Simpler syntax                        | • More boilerplate code    |
| • Uses Hooks like <code>useState</code> | • Uses lifecycle methods   |
| • <hr/>                                 |                            |

## 78) What is Jest?

- Jest is a JavaScript testing framework used for testing React applications.
- ---

## 79) How is React different from Angular?

- |                        |                        |
|------------------------|------------------------|
| • <b>React</b>         | • <b>Angular</b>       |
| • Library              | • Full Framework       |
| • Uses Virtual DOM     | • Uses Real DOM        |
| • One-way data binding | • Two-way data binding |
| • <hr/>                |                        |

## 80) What are the different types of Hooks in React?

- **Basic Hooks:**
- `useState()`
- `useEffect()`
- `useContext()`

- **Additional Hooks:**

- `useReducer()`
- `useRef()`
- `useMemo()`
- `useCallback()`
- `useLayoutEffect()`

- ---

**81) Explain the steps to create a React application and print Hello World**

- Install Node.js.
- Create a React app:
- `npx create-react-app my-app`
- Move to the project folder:
- `cd my-app`
- Run the app:
- `npm start`

- ---

**82) How are comments written in React?**

- Single-line:
- `{/* This is a comment */}`
- Multi-line:
- `{/*  
Multi-line  
Comment  
*/}`

- ---

**83) Explain the concept of Lifting State Up**

- Lifting State Up means moving state from child components to their common parent to share data between children.

- ---

#### 84) What is the purpose of `setState` in React?

- `setState()` updates component state and triggers a re-render.
- \_\_\_\_\_

#### 85) Explain the `useEffect` Hook

- `useEffect` is used to perform side effects such as API calls, subscriptions, or timers.
- Example:
- ```
useEffect(() => {  
  fetchData();  
}, []);
```
- _____

86) What are Controlled Components in React?

- Controlled components are form elements whose values are controlled by React state.
- Example:
- `<input value={name} onChange={handleChange} />`
- _____

87) What is the purpose of the `useReducer` Hook?

- `useReducer` is used for managing complex state logic in React applications.
- _____

88) What is the significance of the `key` attribute in React lists?

- The `key` prop uniquely identifies list items and helps React efficiently update the UI.
- _____

89) What is the difference between Class Components and Functional Components?

- | | |
|--------------------------|-------------------------------|
| • Class Component | • Functional Component |
| • Uses classes | • Uses functions |
| • Uses lifecycle methods | • Uses Hooks |

- **Class Component**
- **Functional Component**
- More code
- Less code

90) Explain the concept of Refs in React

- Refs provide direct access to DOM elements or React components.
- Example:
- `const inputRef = useRef();`

91) Explain the purpose of the useContext Hook

- useContext allows functional components to consume Context values directly.
- Example:
- `const theme = useContext(ThemeContext);`

92) What is the significance of dangerouslySetInnerHTML in React?

- It allows rendering raw HTML inside a component. It should be used carefully to avoid XSS attacks.
- Example:
- `<div dangerouslySetInnerHTML={{ __html: htmlContent }} />`

93) What is the purpose of the componentDidMount lifecycle method?

- `componentDidMount()` is called after a component is mounted and is commonly used for API calls and initialization.

94) What is the React Developer Tool?

- React Developer Tools is a browser extension used to inspect and debug React component trees.

95) How does React handle prop drilling, and how can it be avoided?

- Prop drilling occurs when props are passed through multiple components. It can be avoided using:

- Context API
- Redux
- Zustand

- ---

96) What is the purpose of the shouldComponentUpdate method?

- shouldComponentUpdate() controls whether a component should re-render, helping improve performance.
- Example:
- ```
shouldComponentUpdate(nextProps, nextState) {
 return true;
}
```

**97) What is the significance of the key prop in React Router?**

- The key prop forces a component to remount when its value changes, ensuring the component is reinitialized.

- ---

**98) What is the purpose of the forwardRef function in React?**

- forwardRef allows parent components to pass refs to child components and access their DOM elements directly.
- Example:
- ```
const Input = React.forwardRef((props, ref) => (  
  <input ref={ref} />  
));
```

- ---

99) Explain the concept of Error Boundaries in React

- Error Boundaries are components that catch JavaScript errors in child components and display a fallback UI instead of crashing the entire application.

- ---

100) What is the significance of the memo function in React?

- React.memo() prevents unnecessary re-renders by rendering a component only when its props change.
- Example:

- export default React.memo(MyComponent);

- ---

101) How does React handle forms?

- React handles forms using **controlled components**, where form data is managed by React state.
- Example:
- `<input value={name} onChange={handleChange} />`

- ---

102) Explain the purpose of the useMemo Hook

- useMemo memoizes the result of expensive calculations and recomputes it only when dependencies change.
- Example:
- `const total = useMemo(() => calculateTotal(items), [items]);`

- ---

103) What is the significance of the useCallback Hook?

- useCallback memoizes functions, preventing unnecessary recreation of functions on every render.
- Example:
- `const handleClick = useCallback(() => {
 console.log("Clicked");
}, []);`

- ---

104) What are React Portals?

- React Portals allow components to render children outside the parent DOM hierarchy.
- Common use cases:
- Modals
- Tooltips
- Popups

- ---

105) Explain the concept of Suspense in React

- Suspense lets React display a fallback UI while waiting for components or data to load.
- Example:
- ```
<Suspense fallback={<Loader />}>
 <Profile />
</Suspense>
```
- ---

### 106) What is the purpose of the useEffect cleanup function?

- The cleanup function is used to clean up resources when a component unmounts.
- Examples:
- Remove event listeners
- Clear timers
- Cancel subscriptions
- Example:
- ```
useEffect(() => {  
  const timer = setInterval(() => {}, 1000);  
  
  return () => clearInterval(timer);  
, []);
```
- ---

107) How does React handle routing?

- React handles routing using libraries like **React Router**, which enables navigation between pages without reloading the browser.
- ---

108) What is the purpose of the useEffect Hook?

- useEffect runs synchronously after DOM updates but before the browser repaints the screen.
- It is mainly used for:
- Measuring DOM elements

- Updating layout before paint

- ---

109) What is the significance of the React.memo function?

- React.memo optimizes performance by preventing unnecessary re-renders if props remain unchanged.

- ---

110) How does React handle code splitting?

- React supports code splitting using React.lazy() and Suspense, allowing components to load only when needed.
- Example:
- `const Home = React.lazy(() => import("./Home"));`

- ---

111) What is the purpose of the useImperativeHandle Hook?

- useImperativeHandle customizes the value exposed to parent components when using forwardRef.

- ---

112) Explain the concept of the useDebugValue Hook

- useDebugValue displays custom labels for custom Hooks in React Developer Tools, making debugging easier.
- Example:
- `useDebugValue(isOnline ? "Online" : "Offline");`

113) Explain the significance of the SuspenseList component in React

- SuspenseList coordinates the loading order of multiple Suspense components, providing a better user experience.

- ---

114) What is the significance of the react-scripts package in a React application?

- react-scripts contains configuration and scripts used by Create React App for development, testing, and building applications.
- Commands:

- npm start
npm test
npm run build

- ---

115) What is Middleware in Redux?

- Middleware is a function that sits between dispatching an action and the reducer. It is used for logging, API calls, and handling asynchronous operations.
- Examples:
- Redux Thunk
- Redux Saga

- ---

116) Explain Data Flow in Redux

- Redux follows a **unidirectional data flow**:
- **Action → Reducer → Store → UI Update**
- User dispatches an action.
- Reducer processes the action.
- Store updates the state.
- UI re-renders.

- ---

117) What is Redux Thunk?

- Redux Thunk is middleware that allows action creators to return functions instead of action objects, making asynchronous operations possible.
- Example:
- ```
const fetchUsers = () => async (dispatch) => {
 const data = await api.getUsers();
 dispatch({ type: "GET_USERS", payload: data });
};
```

- ---

### 118) What is Redux Saga? Difference between Redux Thunk and Redux Saga?

- **Redux Saga** is middleware that uses generator functions to manage side effects like API calls.
- **Redux Thunk**
  - Simpler
  - Uses functions
  - Suitable for small apps
- **Redux Saga**
  - More powerful
  - Uses generators
  - Suitable for complex apps

---

#### 119) What is useRef in ReactJS?

- useRef is a Hook used to access DOM elements and store mutable values without causing re-renders.
- Example:
- `const inputRef = useRef(null);`

---

#### 120) What is Server-Side Rendering (SSR) in ReactJS?

- SSR is the process of rendering React components on the server before sending HTML to the browser.
- Benefits:
- Better SEO
- Faster initial page load
- Example: **Next.js**

---

#### 121) What is Server-Side Rendering (SSR) in ReactJS?

- Same as above.
- **One-line Answer:**  
SSR renders React components on the server and sends ready-to-display HTML to the browser.

---

#### 122) What is node\_modules in ReactJS?

- node\_modules is a directory that stores all installed project dependencies and packages.

- ---

### 123) What is the default localhost server port in ReactJS? How can we change it?

- **Default Port:** 3000
- To change it, create a .env file:
- PORT=5000

- ---

### 124) How to optimize a ReactJS application?

- Use React.memo
- Use useMemo and useCallback
- Implement code splitting (React.lazy)
- Use pagination or virtualization
- Avoid unnecessary re-renders

- ---

### 125) What is Context API in ReactJS?

- Context API is React's built-in state management solution for sharing global data without prop drilling.
- Example:
- ```
<UserContext.Provider value={user}>  
  <App />  
</UserContext.Provider>
```

- ---

126) What are super, constructor, and render functions in ReactJS?

- **constructor():** Initializes state and binds methods.
- **super():** Calls the parent class constructor.
- **render():** Returns JSX to display UI.
- Example:

- ```
class App extends React.Component {
 constructor(props) {
 super(props);
 }

 render() {
 return <h1>Hello</h1>;
 }
}
```

- ---

### 127) What is ReactJS and how does it differ from other JavaScript libraries?

- React is a JavaScript library for building user interfaces using reusable components and Virtual DOM.
  - **Differences:**
  - Uses Virtual DOM
  - Component-based architecture
  - Faster rendering
  - One-way data flow
- ---

### 128) What are the advantages of using ReactJS?

- Reusable components
  - Fast rendering using Virtual DOM
  - Easy to maintain
  - Strong community support
  - SEO-friendly with SSR
- ---

### 129) How does ReactJS handle updates and rendering?

- React uses the **Virtual DOM** and **Reconciliation** process to compare changes and update only the necessary parts of the Real DOM, improving performance.

### 130) What are Components in React.js and how are they used?

- Components are reusable building blocks of a React application used to create UI.
- Types:
- Functional Components
- Class Components

---

- 

### **131) What is JSX and how is it used in React.js?**

- JSX (JavaScript XML) allows developers to write HTML-like syntax inside JavaScript to describe the UI.
- Example:
- `const element = <h1>Hello React</h1>;`

---

- 

### **132) How do you use Event Handling in React.js?**

- Event handling in React is done using camelCase event names and functions.
- Example:
- `<button onClick={handleClick}>Click Me</button>`

---

- 

### **133) What is the significance of Props in React.js?**

- Props are used to pass data from parent components to child components.
- Example:
- `<User name="Shilu" />`

---

- 

### **134) How do you use Forms and Form Validation in React.js?**

- Forms are usually handled using controlled components and validated using state or libraries like Formik and Yup.
- Example:
- `<input value={email} onChange={handleChange} />`

---

- 

### **135) How do you use React.js with a state management library such as Redux?**

- Redux is integrated by creating a store and connecting components using hooks like useSelector and useDispatch.
- Example:
- `const count = useSelector(state => state.count);`  
`const dispatch = useDispatch();`
- ---

### 136) How do you use Hooks in React.js?

- Hooks are used inside functional components to manage state and side effects.
- Common Hooks:
- `useState`
- `useEffect`
- `useContext`
- `useRef`
- ---

### 137) How do you use Context API in React.js?

- Context API is used by creating a Context and wrapping components with a Provider.
- Example:
- `<UserContext.Provider value={user}>`  
`<App />`  
`</UserContext.Provider>`
- ---

### 138) How can you optimize the performance of a React.js application?

- Use `React.memo()`
- Use `useMemo()` and `useCallback()`
- Implement code splitting
- Avoid unnecessary re-renders
- Use lazy loading
- ---

### 139) How do you test React.js components?

- React components can be tested using:
- Jest
- React Testing Library
- Example:
- ```
render(<App />);  
expect(screen.getByText("Hello")).toBeInTheDocument();
```

140) How does Server-Side Rendering (SSR) work in React.js?

- SSR renders React components on the server and sends ready HTML to the browser, improving SEO and performance.
- Example: **Next.js**

141) How does React.js handle different types of errors?

- React handles errors using:
- Error Boundaries
- Try-Catch blocks
- Console logging
- Fallback UI

142) Explain the concept of "Lifting State Up" in React and why it's important.

- Lifting State Up means moving shared state to the nearest common parent component so multiple child components can access and update it.
- Importance:
- Enables data sharing between components.
- Maintains a single source of truth.

143) Explain the use of Redux with React and how it differs from React's built-in state management.

- Redux provides centralized state management for large applications, while React's built-in state (useState, Context API) is suitable for local or simple global state.

-

144) Difference between React's Built-in State Management and Redux

- | • React State (useState, Context) | • Redux |
|--|-----------------------------------|
| • Suitable for small applications | • Suitable for large applications |
| • Built into React | • External library |
| • Easy to set up | • More complex setup |
| • Limited features | • Powerful middleware support |

145) Can you explain the concept of "Reactive Updates" in React and how it differs from traditional data binding?

Reactive Updates in React mean that whenever the **state or props change**, React automatically updates and re-renders only the affected components.

How it works:

1. State/Props change.
2. React creates a new Virtual DOM.
3. React compares it with the previous Virtual DOM (**Diffing**).
4. Only the changed parts are updated in the Real DOM.

Difference from Traditional Data Binding

React (Reactive Updates) Traditional Data Binding

| | |
|----------------------------|--------------------------------------|
| Uses Virtual DOM | Directly updates Real DOM |
| One-way data flow | Often uses two-way data binding |
| Updates only changed parts | May update the entire UI |
| Faster and more efficient | Can be slower for large applications |

Example:

```
const [count, setCount] = useState(0);
```

```
<button onClick={() => setCount(count + 1)}>
  {count}
</button>
```

When count changes, React automatically updates only that button's text.

146) Can you explain how React handles performance optimization, such as lazy loading and memoization?

React provides several techniques to improve application performance.

1. Lazy Loading

Lazy loading loads components only when they are needed, reducing the initial bundle size.

Example:

```
const Home = React.lazy(() => import('./Home'));
```

```
<Suspense fallback={<div>Loading...</div>}>
  <Home />
</Suspense>
```

2. Memoization

Memoization prevents unnecessary re-renders and recalculations.

Tools used:

- **React.memo()** → Memoizes components.
- **useMemo()** → Memoizes expensive calculations.
- **useCallback()** → Memoizes functions.

Example (React.memo):

```
const User = React.memo(({ name }) => {
  return <h1>{name}</h1>;
});
```

Other React Performance Optimization Techniques:

- Code Splitting (React.lazy)
- Virtualization (react-window, react-virtualized)
- Pagination

- Avoid unnecessary state updates
- Using key props correctly
- Optimizing API calls
- Debouncing and Throttling

🔥 **One-line Interview Answer:**

- **"React optimizes performance using techniques like lazy loading, code splitting, memoization (`React.memo`, `useMemo`, `useCallback`), and Virtual DOM diffing to minimize unnecessary rendering and improve application speed."**